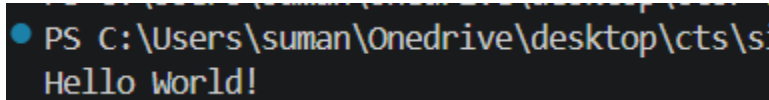


C#

Exercise -1:

```
using System;
class Program{
    static void Main(){
        Console.WriteLine("Hello World!");
    }
}
```

Output:



A screenshot of a PowerShell console window. The prompt is 'PS C:\Users\suman\Onedrive\desktop\cts\s'. The output of the program is 'Hello World!'.

Exercise-2:

```
using System;
class Student{
    public string Name;
}
class Program{
    static void ChangeValue(int x)
    {
        x = 100;
    }
    static void ChangeReference(Student s){
        s.Name = "Sumana Sakhamuri";
    }
    static void Main(){
        int number = 10;
        Student student = new Student();
        student.Name = "Sumana";
        Console.WriteLine("Before Method Call");
        Console.WriteLine("Number = " + number);
        Console.WriteLine("Student = " + student.Name);
        ChangeValue(number);
        ChangeReference(student);
        Console.WriteLine("\nAfter Method Call");
        Console.WriteLine("Number = " + number);
        Console.WriteLine("Student = " + student.Name);
    }
}
```

Output:

```

PS C:\Users\suman\Onedrive\desktop\
C:\Users\suman\Onedrive\desktop\cts
er adding the 'required' modifier o
Before Method Call
Number = 10
Student = Sumana

After Method Call
Number = 10
Student = Sumana Sakhamuri

```

Exercise-3:

```

using System;
class Person(string name, int age)
{
    public string Name { get; } = name;
    public int Age { get; } = age;

    public void Display()
    {
        Console.WriteLine($"Name : {Name}");
        Console.WriteLine($"Age : {Age}");
    }
}
class Program
{
    static void Main()
    {
        Person p = new Person("Sumana", 21);
        p.Display();
    }
}

```

Output:

```

PS C:\Users\suman\Onedrive\desktop\
Name : Sumana
Age : 21

```

Exercise-4:

using System;

```

class Student
{
    public string Name { get; set; }

    public Student(string name)
    {
        Name = name;
    }
}

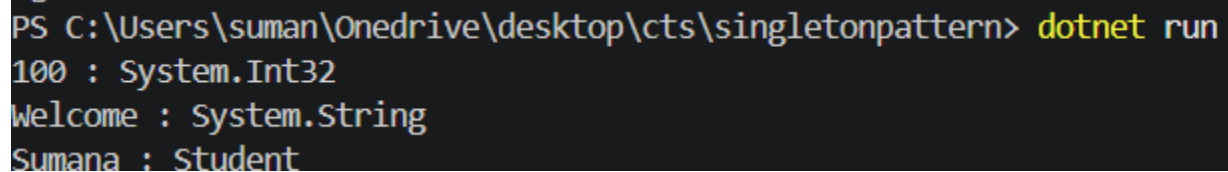
class Program
{
    static void Main()
    {
        var number = 100;
        var message = "Welcome";

        Student s = new("Sumana");

        Console.WriteLine($"{number} : {number.GetType()}");
        Console.WriteLine($"{message} : {message.GetType()}");
        Console.WriteLine($"{s.Name} : {s.GetType()}");
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\cts\singletonpattern> dotnet run
100 : System.Int32
Welcome : System.String
Sumana : Student

```

Exercise-5:

using System;

```

class Program
{
    static void Main()
    {
        Console.Write("Enter Marks : ");
        int marks = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("\nUsing if-else");

        if (marks >= 90)

```

```

        Console.WriteLine("Grade A");
    else if (marks >= 80)
        Console.WriteLine("Grade B");
    else if (marks >= 70)
        Console.WriteLine("Grade C");
    else if (marks >= 60)
        Console.WriteLine("Grade D");
    else
        Console.WriteLine("Fail");

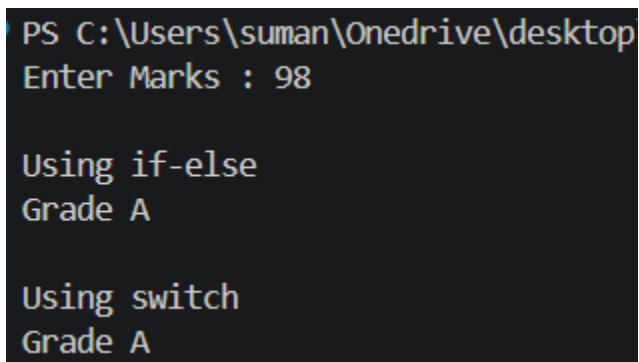
    Console.WriteLine("\nUsing switch");

    string grade = marks switch
    {
        >= 90 => "Grade A",
        >= 80 => "Grade B",
        >= 70 => "Grade C",
        >= 60 => "Grade D",
        _ => "Fail"
    };

    Console.WriteLine(grade);
}
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop
Enter Marks : 98

Using if-else
Grade A

Using switch
Grade A

```

Exercise-6:

using System;

class Program

```

{
    static void Main()
    {
        int[] numbers = { 10, 20, 30, 40, 50 };
    }
}

```

```

Console.WriteLine("Using for loop");
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] == 30)
        continue;

    Console.Write(numbers[i] + " ");
}

Console.WriteLine("\n\nUsing foreach loop");
foreach (int n in numbers)
{
    if (n == 40)
        break;

    Console.Write(n + " ");
}

Console.WriteLine("\n\nUsing while loop");
int j = 0;
while (j < numbers.Length)
{
    Console.Write(numbers[j] + " ");
    j++;
}

Console.WriteLine("\n\nUsing do-while loop");
int k = 0;
do
{
    Console.Write(numbers[k] + " ");
    k++;
} while (k < numbers.Length);
}

```

Output:

```
PS C:\Users\suman\Onedrive\desk
Using for loop
10 20 40 50

Using foreach loop
10 20 30

Using while loop
10 20 30 40 50

Using do-while loop
10 20 30 40 50
```

Exercise-7:
using System;

```
class Calculator
{
    public int CalculateTotal(int a, int b)
    {
        return a + b;
    }

    public double CalculateTotal(double a, double b, double c)
    {
        return a + b + c;
    }
}

class Program
{
    static void Main()
    {
        Calculator obj = new Calculator();

        Console.WriteLine("Integer Total = " + obj.CalculateTotal(10, 20));
        Console.WriteLine("Double Total = " + obj.CalculateTotal(10.5, 20.2, 30.3));
    }
}
```

Output:

```
PS C:\Users\suman\Onedrive
Integer Total = 30
Double Total = 61
```

Exercise-8:
using System;

```
class Program
{
    static void RefMethod(ref int x)
    {
        x += 10;
    }

    static void OutMethod(out int y)
    {
        y = 50;
    }

    static void InMethod(in int z)
    {
        Console.WriteLine("Value received using in = " + z);
    }

    static void Main()
    {
        int a = 20;
        Console.WriteLine("Before ref = " + a);
        RefMethod(ref a);
        Console.WriteLine("After ref = " + a);

        int b;
        OutMethod(out b);
        Console.WriteLine("Out Value = " + b);

        int c = 100;
        InMethod(in c);
    }
}
```

Output:

```

PS C:\Users\suman\Onedrive\desktop\c
Before ref = 20
After ref = 30
Out Value = 50
Value received using in = 100

```

Exercise-9:
using System;

```

class Program
{
    static int CalculateFactorial(int n)
    {
        int Factorial(int x)
        {
            if (x == 0 || x == 1)
                return 1;

            return x * Factorial(x - 1);
        }

        return Factorial(n);
    }

    static void Main()
    {
        int num = 5;

        Console.WriteLine("Factorial = " + CalculateFactorial(num));
    }
}

```

Output:

```

PS C:\Users\suman\Onedrive\desktop\c
Factorial = 120

```

Exercise-10:
using System;

```

class Car
{
    public string Make;
    public string Model;
    public int Year;
}

```



```

// Default Constructor
public Car()
{
    Make = "Toyota";
    Model = "Corolla";
    Year = 2024;
}

// Parameterized Constructor
public Car(string make, string model, int year)
{
    Make = make;
    Model = model;
    Year = year;
}

public void Display()
{
    Console.WriteLine($"Make : {Make}");
    Console.WriteLine($"Model: {Model}");
    Console.WriteLine($"Year : {Year}");
    Console.WriteLine();
}
}

class Program
{
    static void Main()
    {
        Car car1 = new Car();

        Car car2 = new Car("Honda", "City", 2023);

        Console.WriteLine("Default Constructor");
        car1.Display();

        Console.WriteLine("Parameterized Constructor");
        car2.Display();
    }
}

```

Output:

```
PS C:\Users\suman\Onedrive\desкто
Default Constructor
Make : Toyota
Model: Corolla
Year : 2024

Parameterized Constructor
Make : Honda
Model: City
Year : 2023
```

Exercise-11:
using System;

```
class Parent
{
    public string PublicMessage = "Public Member";

    private string PrivateMessage = "Private Member";

    protected string ProtectedMessage = "Protected Member";

    public void ShowPrivate()
    {
        Console.WriteLine(PrivateMessage);
    }
}

class Child : Parent
{
    public void Display()
    {
        Console.WriteLine(PublicMessage);
        Console.WriteLine(ProtectedMessage);
    }
}

class Program
{
    static void Main()
    {
        Parent p = new Parent();
```

```

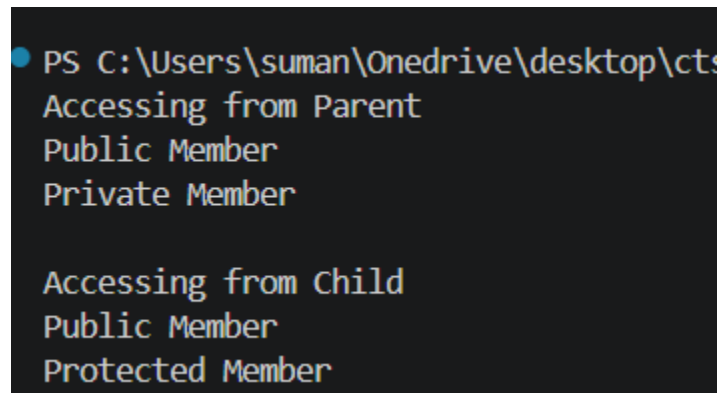
        Console.WriteLine("Accessing from Parent");
        Console.WriteLine(p.PublicMessage);
        p.ShowPrivate();

        Child c = new Child();

        Console.WriteLine("\nAccessing from Child");
        c.Display();
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\cts>
Accessing from Parent
Public Member
Private Member

Accessing from Child
Public Member
Protected Member

```

Exercise-12:
using System;

```

class Product
{
    public string Name { get; set; }

    private double price;

    public double Price
    {
        get { return price; }
        set
        {
            if (value >= 0)
                price = value;
            else
                Console.WriteLine("Price cannot be negative.");
        }
    }
}

```

```

class Program
{
    static void Main()
    {
        Product p = new Product();

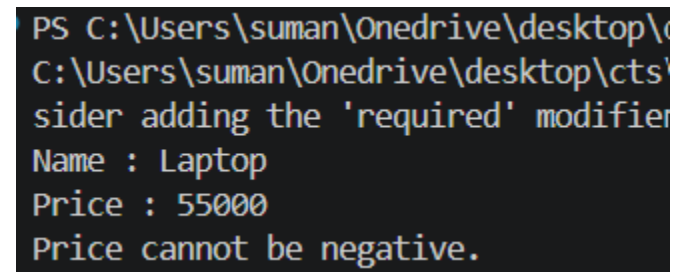
        p.Name = "Laptop";
        p.Price = 55000;

        Console.WriteLine("Name : " + p.Name);
        Console.WriteLine("Price : " + p.Price);

        p.Price = -500;
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\c
C:\Users\suman\Onedrive\desktop\cts
sider adding the 'required' modifier
Name : Laptop
Price : 55000
Price cannot be negative.

```

Exercise:13:

using System;

```

public record Employee
{
    public int Id { get; init; }
    public string Name { get; init; }
    public double Salary { get; init; }
}

```

```

class Program
{
    static void Main()
    {
        Employee emp1 = new Employee
        {
            Id = 101,
            Name = "Alice",
            Salary = 50000
        }
    }
}

```

```

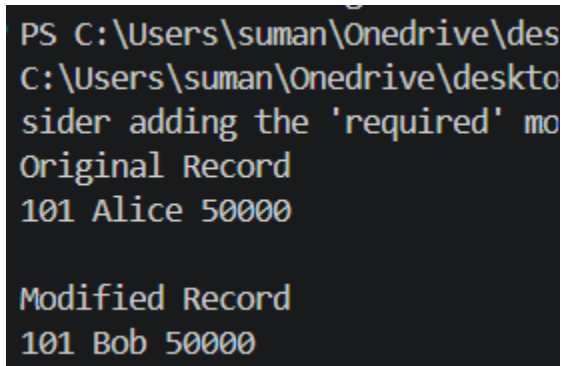
};

Employee emp2 = emp1 with
{
    Name = "Bob"
};

Console.WriteLine("Original Record");
Console.WriteLine($"{emp1.Id} {emp1.Name} {emp1.Salary}");

Console.WriteLine("\nModified Record");
Console.WriteLine($"{emp2.Id} {emp2.Name} {emp2.Salary}");
}
}
Output:

```



```

PS C:\Users\suman\Onedrive\desktop>
C:\Users\suman\Onedrive\desktop>
sider adding the 'required' mo
Original Record
101 Alice 50000

Modified Record
101 Bob 50000

```

Exercise-14:
using System;

```

class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine("Drawing Shape");
    }
}

class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}

```

```

class Rectangle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing Rectangle");
    }
}

```

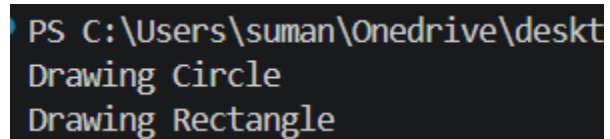
```

class Program
{
    static void Main()
    {
        Shape s1 = new Circle();
        Shape s2 = new Rectangle();

        s1.Draw();
        s2.Draw();
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\deskt
Drawing Circle
Drawing Rectangle

```

Exercise-15:
using System;

```

abstract class Vehicle
{
    public abstract void Drive();
}

```

```

interface IDrivable
{
    void Start();
}

```

```

class Car : Vehicle, IDrivable
{
    public override void Drive()
    {
        Console.WriteLine("Car is Driving");
    }
}

```

```

    }

    public void Start()
    {
        Console.WriteLine("Car Started");
    }
}

```

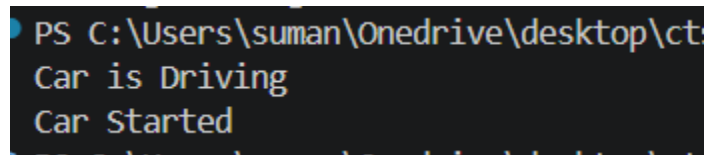
```

class Program
{
    static void Main()
    {
        Vehicle v = new Car();
        v.Drive();

        IDrivable d = new Car();
        d.Start();
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\ct
Car is Driving
Car Started

```

Exercise-16:
using System;

```

class Person
{
    public string? Name { get; set; }
}

```

```

class Program
{
    static void Main()
    {
        Person? p = null;

        Console.WriteLine("Using Null-Conditional Operator:");
        Console.WriteLine(p?.Name);

        p = new Person();
    }
}

```

```

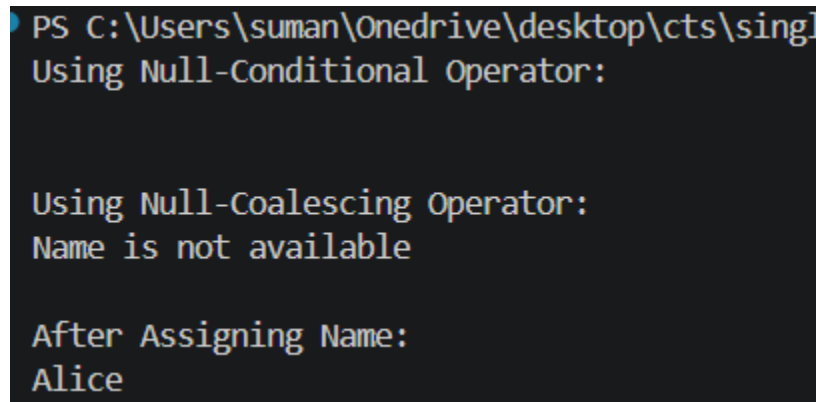
        Console.WriteLine("\nUsing Null-Coalescing Operator:");
        Console.WriteLine(p.Name ?? "Name is not available");

        p.Name = "Alice";

        Console.WriteLine("\nAfter Assigning Name:");
        Console.WriteLine(p.Name);
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\cts\singl
Using Null-Conditional Operator:

Using Null-Coalescing Operator:
Name is not available

After Assigning Name:
Alice

```

Exercise-17:
using System;

```

class Contact
{
    public string? Name { get; set; }
    public string? PhoneNumber { get; set; }
}

class Program
{
    static void Main()
    {
        Contact? contact = new Contact();

        Console.WriteLine(contact?.Name ?? "No Contact Name");

        contact.Name = "Bob";
        contact.PhoneNumber = "9876543210";

        Console.WriteLine("Name : " + contact?.Name);
    }
}

```

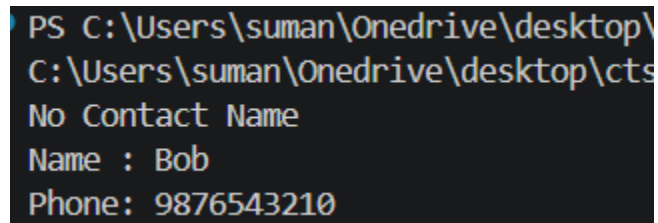


```

        Console.WriteLine("Phone: " + contact?.PhoneNumber);
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\
C:\Users\suman\Onedrive\desktop\cts
No Contact Name
Name : Bob
Phone: 9876543210

```

Exercise-18:
using System;

```

class Student
{
    public required int Id { get; set; }

    public required string Name { get; set; }

    public int Marks { get; set; }
}

class Program
{
    static void Main()
    {
        Student s = new Student
        {
            Id = 101,
            Name = "Alice",
            Marks = 95
        };

        Console.WriteLine("Student Details");
        Console.WriteLine($"ID : {s.Id}");
        Console.WriteLine($"Name : {s.Name}");
        Console.WriteLine($"Marks : {s.Marks}");

        // Uncommenting the code below will cause a compile-time error
        /*
        Student s2 = new Student
        {
            Id = 102

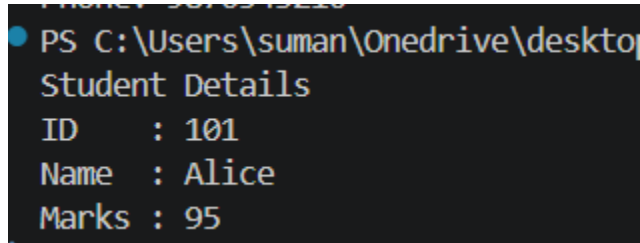
```

```

    };
    */
}
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop
Student Details
ID : 101
Name : Alice
Marks : 95

```

Exercise-19:

```

using System;
using System.Collections.Generic;

```

```

class Program
{
    static void Main()
    {
        List<string> fruits = new List<string>()
        {
            "Apple",
            "Banana",
            "Orange"
        };

        fruits.Add("Mango");
        fruits.Remove("Banana");

        Console.WriteLine("List Elements");

        foreach (string fruit in fruits)
        {
            Console.WriteLine(fruit);
        }

        Dictionary<int, string> students = new Dictionary<int, string>();

        students.Add(101, "Alice");
        students.Add(102, "Bob");
        students.Add(103, "Charlie");

        Console.WriteLine("\nDictionary Elements");
    }
}

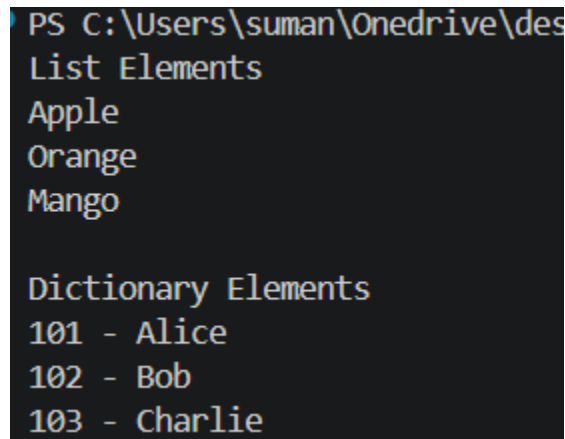
```

```

        foreach (KeyValuePair<int, string> item in students)
        {
            Console.WriteLine($"{item.Key} - {item.Value}");
        }
    }
}

```

Output:



```

PS C:\Users\suman\Onedrive\des
List Elements
Apple
Orange
Mango

Dictionary Elements
101 - Alice
102 - Bob
103 - Charlie

```

Exercise-20:

```

using System;
using System.Collections.Generic;
using System.Linq;

```

```

class Order
{
    public int OrderId { get; set; }
    public string CustomerName { get; set; } = "";
    public double TotalAmount { get; set; }
}

```

```

class Program
{
    static void Main()
    {
        List<Order> orders = new List<Order>()
        {
            new Order { OrderId = 1, CustomerName = "Alice", TotalAmount = 5000 },
            new Order { OrderId = 2, CustomerName = "Bob", TotalAmount = 2500 },
            new Order { OrderId = 3, CustomerName = "Charlie", TotalAmount = 8000 },
            new Order { OrderId = 4, CustomerName = "David", TotalAmount = 1500 }
        };

        var result = orders
    }
}

```

```

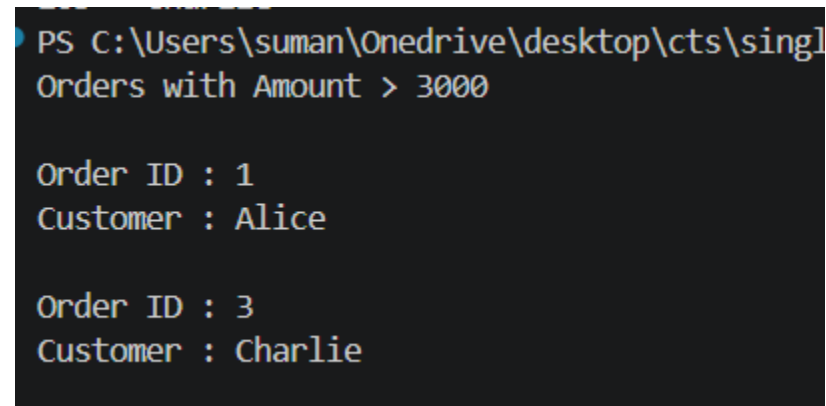
        .Where(o => o.TotalAmount > 3000)
        .Select(o => new
        {
            o.OrderId,
            o.CustomerName
        });

Console.WriteLine("Orders with Amount > 3000\n");

foreach (var order in result)
{
    Console.WriteLine($"Order ID : {order.OrderId}");
    Console.WriteLine($"Customer : {order.CustomerName}");
    Console.WriteLine();
}
}
}

```

Output:



```

PS C:\Users\suman\Onedrive\desktop\cts\singl
Orders with Amount > 3000

Order ID : 1
Customer : Alice

Order ID : 3
Customer : Charlie

```

Exercise-21:
using System;

```

class Program
{
    static void DisplayInfo(object obj)
    {
        // Pattern Matching using 'is'
        if (obj is int number)
        {
            Console.WriteLine($"Integer Value : {number}");
        }
        else if (obj is string text)
        {
            Console.WriteLine($"String Value : {text}");
        }
    }
}

```

```

    }
    else
    {
        Console.WriteLine("Unknown Type");
    }

    // Pattern Matching using switch
    switch (obj)
    {
        case int n:
            Console.WriteLine($"Square = {n * n}");
            break;

        case string s:
            Console.WriteLine($"Length = {s.Length}");
            break;

        default:
            Console.WriteLine("No Operation");
            break;
    }
}

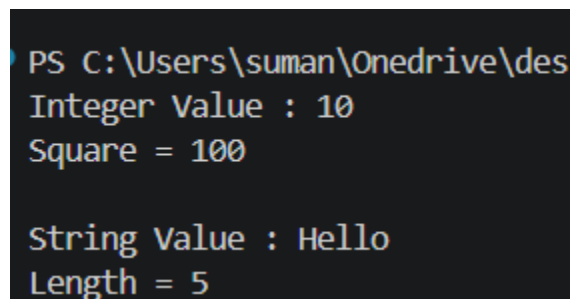
static void Main()
{
    DisplayInfo(10);

    Console.WriteLine();

    DisplayInfo("Hello");
}
}

```

Output:



```

PS C:\Users\suman\Onedrive\des
Integer Value : 10
Square = 100

String Value : Hello
Length = 5

```

Exercise-22:
using System;

```

class Program
{
    static (int, string) GetStudent()
    {
        return (101, "Alice");
    }

    static void Main()
    {
        (int id, string name) = GetStudent();

        Console.WriteLine("Student ID : " + id);
        Console.WriteLine("Student Name : " + name);
    }
}

```

Output:

```

PS C:\Users\suman\Onedrive\de
Student ID : 101
Student Name : Alice

```

Exercise-23:

```

using System;
using System.Threading.Tasks;

```

```

class Program
{
    static async Task<string> UploadFileAsync()
    {
        await Task.Delay(3000);

        return "File Uploaded Successfully";
    }

    static async Task Main()
    {
        try
        {
            Console.WriteLine("Uploading File...");

            string message = await UploadFileAsync();

            Console.WriteLine(message);
        }
    }
}

```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}

```

Output:

```

PS C:\Users\suman\Onedrive\desktop\
Uploading File...
File Uploaded Successfully

```

Exercise-24:

```

using System;
using System.IO;
using System.Text.Json;

```

```

class User
{
    public string Name { get; set; }
    public int Age { get; set; }
    public string Email { get; set; }
}

```

```

class Program
{
    static void Main()
    {
        User user = new User
        {
            Name = "Sumana",
            Age = 21,
            Email = "sumana@gmail.com"
        };

        string json = JsonSerializer.Serialize(user, new JsonSerializerOptions
        {
            WriteIndented = true
        });

        File.WriteAllText("user.json", json);

        Console.WriteLine("JSON Saved Successfully");
    }
}

```

```

string readJson = File.ReadAllText("user.json");

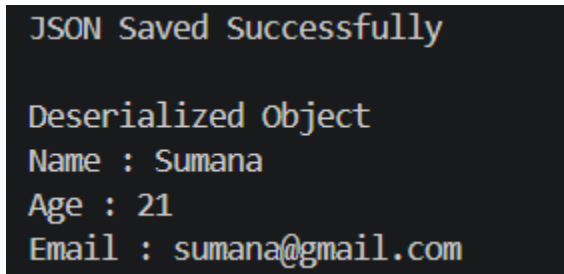
User newUser = JsonSerializer.Deserialize<User>(readJson);

Console.WriteLine("\nDeserialized Object");

Console.WriteLine("Name : " + newUser.Name);
Console.WriteLine("Age : " + newUser.Age);
Console.WriteLine("Email : " + newUser.Email);
}
}

```

Output:



```

JSON Saved Successfully

Deserialized Object
Name : Sumana
Age : 21
Email : sumana@gmail.com

```

Exercise-25:

```

using System;
using System.IO;
using System.Text;

```

```

class Program
{
    static void Main()
    {
        // Create a sample file
        File.WriteAllText("sample.txt", "Welcome to C# Programming");

        // Read using FileStream
        using (FileStream fs = new FileStream("sample.txt", FileMode.Open, FileAccess.Read))
        {
            byte[] buffer = new byte[fs.Length];

            fs.Read(buffer, 0, buffer.Length);

            string content = Encoding.UTF8.GetString(buffer);

            Console.WriteLine("File Content");
            Console.WriteLine(content);
        }
    }
}

```



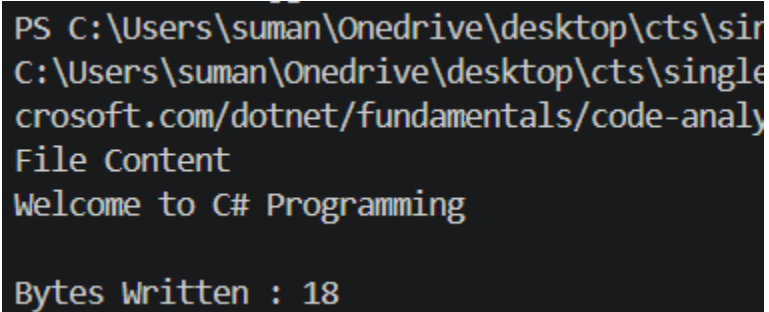
```
// Write using MemoryStream
using (MemoryStream ms = new MemoryStream())
{
    byte[] data = Encoding.UTF8.GetBytes("Hello MemoryStream");

    ms.Write(data, 0, data.Length);

    Console.WriteLine();

    Console.WriteLine("Bytes Written : " + ms.Length);
}
}
```

Output:



```
PS C:\Users\suman\Onedrive\desktop\cts\sin
C:\Users\suman\Onedrive\desktop\cts\single
crosoft.com/dotnet/fundamentals/code-analy
File Content
Welcome to C# Programming

Bytes Written : 18
```